Available online at www.sciencedirect.com

ScienceDirect

IFAC PapersOnline CONFERENCE PAPER ARCHIVE

IFAC PapersOnLine 54-20 (2021) 692–698

Non-Autoregressive vs Autoregressive Neural Networks for System Identification ⋆

Daniel Weber * **Clemens Gühmann** **

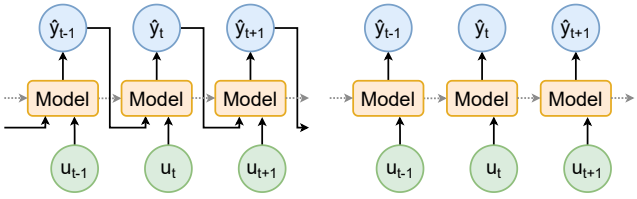
* *Technische Universität Berlin, Berlin, 10587 Germany (e-mail: d.weber.1@tu-berlin.de).*

** *Technische Universität Berlin, Berlin, 10587 Germany (e-mail: clemens.guehmann@tu-berlin.de)*

Abstract: The application of neural networks to non-linear dynamic system identification tasks has a long history, which consists mostly of autoregressive approaches. Autoregression, the usage of the model outputs of previous time steps, is a method of transferring a system state between time steps, which is not necessary for modeling dynamic systems with modern neural network structures, such as gated recurrent units (GRUs) and Temporal Convolutional Networks (TCNs). We compare the accuracy and execution performance of autoregressive and non-autoregressive implementations of a GRU and TCN on the simulation task of three publicly available system identification benchmarks. Our results show, that the non-autoregressive neural networks are significantly faster and at least as accurate as their autoregressive counterparts. Comparisons with other state-of-the-art black-box system identification methods show, that our implementation of the non-autoregressive GRU is the best performing neural network-based system identification method, and in the benchmarks without extrapolation, the best performing black-box method.

Copyright © 2021 The Authors. This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0/)

Keywords: Nonlinear System Identification, Neural Networks, Gated Recurrent Unit, Temporal Convolutional Network, Benchmark Systems, Wiener-Hammerstein, Process Noise



(a) Autoregressive model (b) Non-autoregressive model

Fig. 1. Signal flow of an autoregressive and a non-autoregressive model with an optional internal state for three time steps.

1. INTRODUCTION

System identification is essential for many tasks, such as system control and sensor fusion. It has a long history, beginning with the identification of linear systems (Zadeh, 1956). Most practical relevant systems are nonlinear and dynamic, which led to the development of nonlinear dynamic system identification methods.

Those models may be autoregressive or non-autoregressive, as visualized in Figure 1. The output of an autoregressive model is dependant on the model outputs of previous time steps, which is not the case for a non-autoregressive model. In both cases, the model may have an internal state, which is transferred between each time step. An example for an autoregressive model without an internal state is the nonlinear autoregressive exogenous (NARX) model and for a non-autoregressive model with an internal state is the non-linear state space (NLSS) model.

The identified models may be used for prediction or simulation. Prediction is the task of estimating a limited amount of time steps ahead with information of the past system outputs. Simulation is the task of estimating the system output only with the inputs. In the present work, we will focus only on the simulation task.

Neural networks have been applied to system identification tasks for a long time. Historically feedforward neural networks are applied autoregressively to model the system dynamics, which is a variant of a NARX model (CHEN et al., 1990). The success of this approach was limited by the available hardware and the gradient propagation over long sequences. With the recent development of deep learning-based methods in computer vision and natural language processing, more sophisticated software and hardware for the training of neural networks became available.

With this development, the application of neural networks for system identification has become widespread, with a focus on improving upon existing black-box system identification methods. Most current neural network-based system identification methods still are NARX variants with different neural network architectures as nonlinearities. In related work, a multitude of system identification methods that are based on autoregressive neural networks has been proposed using multilayer perceptrons (MLPs) (Shi et al., 2019), cascaded MLPs (Ljung et al., 2020), convolutional neural networks (Lopez and Yu, 2017), TCNs (Andersson

⋆ This work was partly funded by the German Federal Ministry of Education and Research (FKZ: 16EMO0262).

2405-8963 Copyright © 2021 The Authors. This is an open access article under the CC BY-NC-ND license.
Peer review under responsibility of International Federation of Automatic Control.
10.1016/j.ifacol.2021.11.252

et al., 2019), and recurrent neural networks (Kumar et al., 2019) with promising results.

Although neural networks that are based on convolutional or recurrent layers have inherent capabilities to model dynamic systems, only a limited amount of work omitted the autoregression. It has been shown, that RNNs behave like an NLSS (Ljung et al., 2020) and an RNN has been applied non-autoregressively to a synthetic dataset (Gonzalez and Yu, 2018). The authors of the present paper applied non-autoregressive TCNs and gated recurrent units (GRUs) to an inertial measurement-based sensor fusion task, outperforming state-of-the-art domain-specific sensor fusion methods (Weber et al., 2020).

To the best of our knowledge, it has not yet been analyzed what the differences in the implementation, accuracy, and execution performance of autoregressive and non-autoregressive neural networks are.

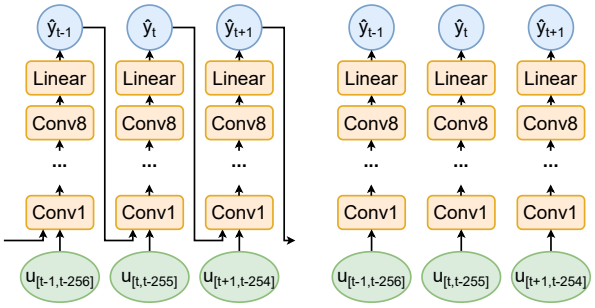
The present paper focuses on the comparison of autoregressive and non-autoregressive variants of TCNs and GRUs with the following main contributions:

- We describe a workflow for autoregressive and non-autoregressive neural networks for system identification using state-of-the-art deep learning tools and methods.
- We find that non-autoregressive neural networks are faster and easier to implement than their autoregressive counterparts and are as least as accurate.
- We compare the estimation performance of the proposed neural networks and state-of-the-art black-box system identification methods on three publicly available benchmark datasets.
- We find that non-autoregressive gated recurrent units consistently outperform all other neural network-based system identification models and in the benchmarks without extrapolation, all black-box models.

2. NEURAL NETWORK IMPLEMENTATION

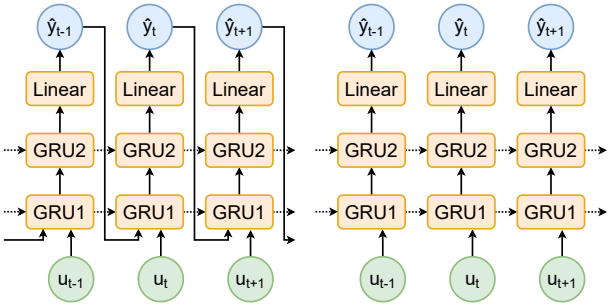
In this section, we describe two different neural network architectures, which have the inherent ability to model dynamic systems without autoregressive connections. Furthermore, we describe the corresponding training process with best practices from sequential data processing with neural networks. The two architectures that we consider are TCNs, which process large sequences at once, and GRUs which propagate hidden states over time.

TCNs are stateless feed-forward networks that apply 1d-dilated causal convolutions to sequences (Andersson et al., 2019), which are inspired by Wavenet, which is used for raw audio generation (Oord et al., 2016). The convolutional layers are stacked on top of each other, which results in a receptive field that describes the number of input samples that are taken into account for the prediction of an output value. The dilation in the convolutions increases the receptive field exponentially with the layer size, enabling the processing of sequences with relations over long periods of time. The main advantage of TCNs is that they are fast in training and inference because of their high parallelizability. The main disadvantage of TCNs is that their ability to model system dynamics is limited by the size of the receptive field, limiting its viability in systems that



(a) TCN-AR (b) TCN-NAR

Fig. 2. Signal flow of the autoregressive TCN (TCN-AR) and non-autoregressive TCN (TCN-NAR) for three time steps with $N_l = 8$.



(a) GRU-AR (b) GRU-NAR

Fig. 3. Signal flow of the autoregressive GRU (GRU-AR) and non-autoregressive GRU (GRU-NAR) for three time steps with $N_l = 2$.

are influenced by integrators over an indefinite amount of time (Weber et al., 2020).

GRUs are a variant of RNNs, that use recurrent connections in their hidden layers to store a hidden state for each time step. This approach has the advantage, that the state information can be theoretically stored over an indefinite amount of time, which in practice is limited by the vanishing gradient problem (Hochreiter, 2011).

The vanishing gradient problem stems from numerical issues on long backpropagation paths in the optimization process. In autoregressive neural networks, the backpropagation path is very long because for every time step the gradient has to propagate from the output through all layers back to the input. In RNNs, the backpropagation path is shorter, but the vanishing gradient problem is still limiting at hundreds of time steps (Hochreiter, 2011).

The gating mechanism of GRUs alleviates this issue, enabling the network to propagate the hidden state over thousands instead of hundreds of time steps during training. Several regularization methods for RNNs have been proposed, that reduce overfitting and improve generalizability (Merity et al., 2017). The main disadvantage of GRUs over TCNs is their sequential nature, which limits its parallelizability, reducing the training speed and especially the inference speed on acceleration hardware.

Both network architectures may be used as autoregressive models, which we refer to as GRU-AR and TCN-AR, and as non-autoregressive models, which we refer to as GRU-

NAR and TCN-NAR. The structures of these models are visualized in Figure 2 and 3. Autoregressive models may be trained using teacher-forcing, with the ground-truth values as input, or in free-running mode, with its own output as input (Lamb et al., 2016). Teacher forcing is faster in training, but only accurate for one-step-ahead prediction tasks, where the model only has to predict the next sample with the values measured in the past. For simulation with neural networks, teacher-forcing is inaccurate, which is why we train the autoregressive models in free-running mode (Ribeiro and Aguirre, 2018).

The naive application of TCN-AR would be very slow and requires an enormous amount of memory during training, because of the number of redundant calculations in its broad receptive field. Because of that, we implemented an optimized caching scheme, which enables us to train TCN-AR in free-running mode in the first place(Paine et al., 2016).

We implemented the training process with FastAI 2, which is a deep learning library that is built upon Pytorch (Howard and Gugger, 2020). For the optimizer, we use the current state-of-the-art combination of RAdam and Lookahead which proved to be effective in various tasks and requires no learning-rate warm-up phase (Liu et al., 2019), (Zhang et al., 2019). Because we need no warm-up phase, we use cosine-annealing for decreasing the learning rate as soon as the optimizer hits an optimization plateau (Loshchilov and Hutter, 2017). We find the maximum learning rate, with which the optimizer begins, with the learning rate finder heuristic (Smith, 2017).

The datasets used for training consist of multiple measured sequences of different lengths. We extract overlapping windows with varying starting points from the sequences with a fixed length N_s for the generation of the mini-batches, to avoid memorizing the sequences. To process longer sequences, we use truncated backpropagation through time (TBPTT), which splits each window into multiple subsequences of length N_{bp} and applies the neural network sequentially (Tallec and Ollivier, 2017). To apply TBPTT to autoregressive models, not only the hidden state but also the last generated output has to be transferred between two mini-batches. The input signals are standardized to zero mean and a standard deviation of one, which is also applied just-in-time to the autoregressive inputs (Ioffe and Szegedy, 2015).

We use the elementwise mean-squared error as the loss function, which is close to the evaluation metric root-mean-square error. Every estimation of dynamic systems that starts with an unknown system state has a transient phase, that describes the time period the estimator needs to converge to a steady-state solution. To minimize the long-term error, we exclude the transition phase from the loss function in the optimization process. With TBPTT, only the first mini-batch of a sequence is affected, because all following mini-batches have a converged system state. TCNs have a receptive field that describes the number of samples that are taken into account for each estimated value. We exclude the estimations that have fewer input values than the receptive field from the gradient propagation because it relies on padded values with no measured information.

The optimization process of the neural network requires a multitude of hyperparameters that have a significant influence on the final performance of the model. Because the hyperparameters span a vast optimization space, it is difficult to find the optimal configuration without an efficient optimization algorithm. For this, we use the Asynchronous Successive Halving Algorithm (ASHA) (Li et al., 2020), a state-of-the-art black-box optimizer that combines random-search with early-stopping in an asynchronous environment.

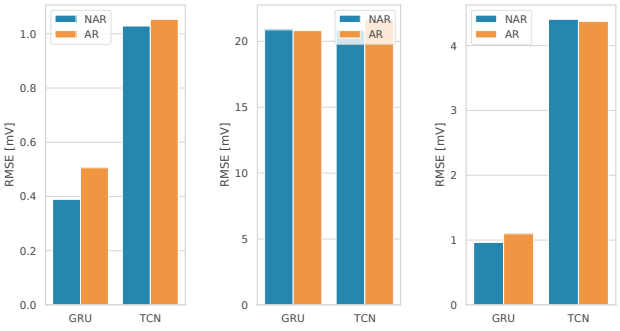
3. PERFORMANCE EVALUATION

In this section, we evaluate the accuracy and the simulation time of the neural network models, which we described in Section 2, on several publicly available datasets. Additionally, we compare the performance of the neural network models with the results of state-of-the-art system identification methods of related work. All models have been trained and executed on an Nvidia RTX 2080 Ti.

The following datasets are designed specifically as benchmarks for non-linear system identification methods:

- Silverbox (Wigren and Schoukens, 2013): The Silverbox benchmark was generated from an electrical circuit that models a nonlinear progressive spring as an oscillating system. The main challenge of the dataset is the extrapolation part of the test sequence, where the output values are larger than any value in the estimation sequence.
- Wiener-Hammerstein (Schoukens et al., 2009): The Wiener Hammerstein benchmark was generated by an electrical circuit that models a Wiener-Hammerstein system.
- Wiener Hammerstein with Process Noise (Schoukens and Noel, 2016): The Wiener-Hammerstein benchmark with process noise was generated by an electrical circuit that models a Wiener-Hammerstein system with additive process noise in the training data. Identifying the process without the process noise is the main challenge of the dataset.

All benchmark datasets provide an estimation and a test subset. For the training and hyperparameter optimization of the neural networks, we split the given estimation subsets further in training and validation subsets. The estimation subset of the Wiener-Hammerstein benchmark with process noise consists of multiple sequences of which we assign randomly 80% to the training and 20% to the validation subset. The Wiener-Hammerstein benchmarks estimation subset consists of a single sequence of which we use the first 80% for training and the remainder for validation. The Silverbox benchmarks estimation subset consists of a single amplitude sweep sequence. A temporal split would lead to extrapolation, which is why we use a random 80/20 split of the extracted windows of the estimation sequence. For the evaluation of the performance of the validated and tested models, we use the root-mean-square error (RMSE). We also ignore the output of the first N samples in the transient phase of the simulation with N given in the description of each benchmark dataset.



(a) WH (b) WH-Noise (c) Silverbox

Fig. 4. RMSE of every model for the test subset of the Wiener-Hammerstein benchmark (a), the Wiener-Hammerstein benchmark with process noise (b), and the Silverbox benchmark (c). Non-autoregressive models perform better than their autoregressive counterparts.

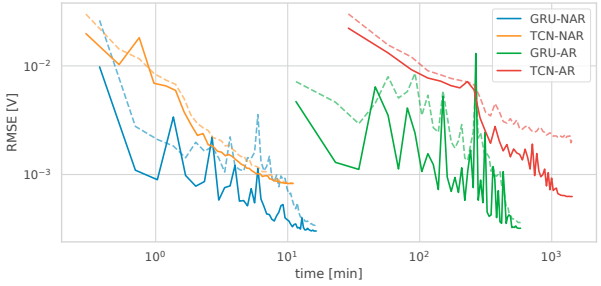


Fig. 5. Validation- and training-RMSE over the duration of the training process for each model on the Silverbox dataset. The training-RMSE is drawn as dashed lines, the validation-RMSE as solid lines. The autoregressive models require significantly more time to achieve similar degrees of performance.

For every dataset, the hyperparameters for the GRU and the TCN have been optimized within the following search space:

(1) Number of neurons per layer: $N_n \in [10, 200]$
(2) Number of hidden layers: $N_l \in [1, 10]$
(3) Mini-batch size: $N_{bs} \in [8, 128]$
(4) Window length: $N_s \in [2000, 20000]$
(5) Sequence length of a mini-batch: $N_{bp} \in [250, 2000]$

Table 1 shows the hyperparameter configurations of each model for every benchmark which are used for the following evaluation.

3.1 Autoregressive vs Non-Autoregressive Neural Networks

First, we compare the accuracy of the autoregressive and non-autoregressive variants of the models on every dataset. Figure 4 visualizes the test RMSE of every model on every dataset. The errors of both variants are close to each other, with the non-autoregressive variant being slightly more accurate. This may be caused by a better fitting set of hyperparameters for the non-autoregressive models.

The hyperparameter optimization process for the autoregressive models is limited by their low training speed. Figure 5 visualizes the evolution of the training and validation RMSE over the time of the training of each model variant on the Silverbox dataset with the individual optimized set of hyperparameters. It becomes clear that the non-autoregressive models finish the training process significantly faster than the autoregressive models. In fact, TCN-AR is so slow, that TCN-NAR and GRU-NAR finish all mini-batches before the first mini-batch of TCN-AR is finished. This advantage in speed allows for more extensive hyperparameter optimizations as well as more complex models with the same amount of resources.

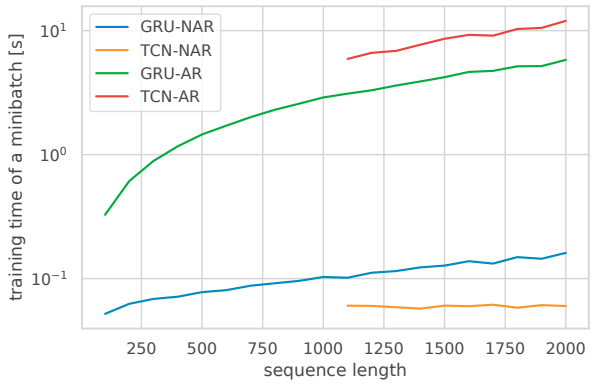


Fig. 6. The training time of a mini-batch with a size of 16 over a variable sequence length. Autoregressive models scale worse than non-autoregressive models with increasing sequence lengths.

All hyperparameters, except the sequence length of the mini-batch, have a similar impact on the training and simulation speed of autoregressive and non-autoregressive models.

Because of their sequential nature, the training speed of the autoregressive models is significantly more affected by the sequence length of the mini-batch than non-autoregressive models are. Figure 6 visualizes the training time of a mini-batch for each model variant over the sequence length. The TCN models require a sequence length of at least 1023 samples because of the receptive field of $2^{10} - 1 = 1023$ samples, which results from the depth of 10, which we identified in the hyperparameter optimization. The training time of the autoregressive models scales much worse with the sequence length than the non-autoregressive models. The training time of TCN-AR is especially bad, considering that we already use an optimized caching algorithm. In contrast, TCN-NAR has over the evaluated value range of sequence lengths a constant training duration, because the convolutions are executed independently over the sequence on the GPU.

The model variants also require different amounts of time for the simulation of sequences with variable lengths. Figure 7 visualizes the inference time of each model variant for the simulation of sequences with variable length. The sequential models GRU-AR, GRU-NAR, and TCN-AR scale linearly with the sequence length while TCN-NAR has a constant inference time over the evaluated range because of its parallel nature.

Table 1. Optimized hyperparameters of each model for every benchmark. (N_n:Neurons per layer, N_l:Hidden layers, N_{bs}:Mini-batch size, N_s:Window length, N_{bp}:Sequence length of a mini-batch)

Benchmark	Model	N_n	N_l	N_{bs}	N_s	N_{bp}	Learnable parameters
Silverbox	GRU-AR	80	1	16	9000	400	26481
	GRU-NAR	100	1	16	9000	500	41101
	TCN-AR	40	10	16	9000	500	29801
	TCN-NAR	50	10	16	9000	500	46251
Wiener-Hammerstein	GRU-AR	180	2	16	9000	400	327061
	GRU-NAR	190	2	16	9000	600	364231
	TCN-AR	80	10	16	9000	500	117201
	TCN-NAR	100	10	16	9000	500	182501
Wiener-Hammerstein with process noise	GRU-AR	190	2	32	9000	500	364231
	GRU-NAR	200	2	32	9000	500	403401
	TCN-AR	90	10	32	9000	500	148051
	TCN-NAR	100	10	32	9000	500	182501

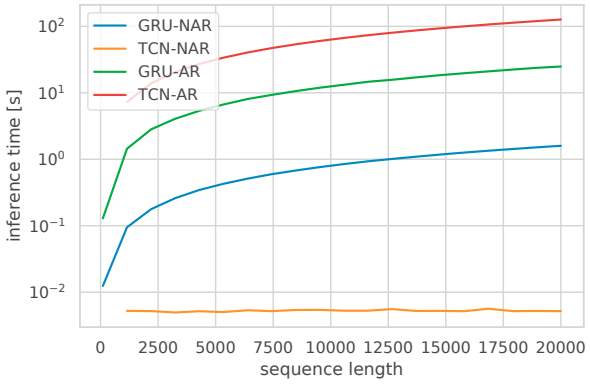


Fig. 7. The inference time of a single sequence with variable sequence length for each model. Autoregressive models are slower than non-autoregressive models.

Table 2. Test RMSE of black-box models in the Silverbox benchmark

RMSE [mV]	Method
0.26	PNLSS (Paduart et al., 2010)
0.33	NN + Cubic Regressor (Ljung et al., 2004)
0.96	**GRU-NAR (NN)**
2.18	TCN (NN) (Maroli and Redmill, 2019)
3.98	LSTM (NN) (Andersson et al., 2019)
4.88	TCN (NN) (Andersson et al., 2019)

All in all, the autoregressive variants of the evaluated networks are slower in training and inference without any benefits for accuracy.

3.2 Comparison with State-of-the-art Methods

In all three benchmarks, GRU-NAR is the most accurate model of the ones, that we implemented. We compare the accuracy of GRU-NAR with the results of state-of-the-art neural network-based and other black-box system identification methods that were applied in related work on these datasets.

The test results are compared in Table 2 for the Silverbox benchmark, Table 3 for the Wiener-Hammerstein benchmark, and Table 4 for the Wiener-Hammerstein benchmark with process noise.

In the Silverbox benchmark, black-box system identification methods have difficulties with the test dataset because of the extrapolation part. The extrapolation performance

Table 3. Test RMSE of black-box models in the Wiener-Hammerstein benchmark

RMSE [mV]	Method
0.39	**GRU-NAR (NN)**
0.42	PNLSS (Paduart et al., 2009)
0.49	Wiener-Hammerstein (Wills and Ninness, 2009)
2.98	BLA-based (Lauwers and Schoukens, 2010)
4.71	FS-LSSVM (De Brabanter et al., 2009)
17.6	CNN (NN) (Lopez and Yu, 2017)
45.61	FFH (NN) (Romero Ugalde et al., 2013)
51.4	DN-BI (NN) (Rosa et al., 2015)

Table 4. Test RMSE of black-box models in the Wiener-Hammerstein benchmark with process noise

RMSE [mV]	Method
20.3	**GRU-NAR (NN)**
25	WH-EIV (Schoukens, 2016)
30	PNLSS (Gedon et al., 2020)
30.3	NFIR (Belz et al., 2017)
42.35	STORN (NN) (Gedon et al., 2020)
54.1	VAE-RNN (NN) (Gedon et al., 2020)

highly depends on the estimator structure and can not generalize over all systems, which is why this is a major shortcoming of neural networks. In this case, the system has a cubic nonlinearity, which is why a combination of a neural network with a cubic regressor, and a polynomial non-linear state-space model perform better in the extrapolation part than more complex models.

In the Wiener-Hammerstein benchmark and the Wiener-Hammerstein benchmark with process noise, GRU-NAR outperforms not only the neural network-based models but all black-box system identification models.

The comparison demonstrates, that the performance of neural network-based models depends to a large extent on the implementation of the structure and training process. This results in GRU-NAR being the best performing neural network-based implementation, which is also highly competitive with other black-box system identification methods.

4. CONCLUSION

In this work, we described the implementation of non-autoregressive and autoregressive neural networks with current best practices for system identification. We compared the accuracy, training, and inference performance of

the different models on three publicly available benchmark datasets. Finally, we compared the accuracy of the best performing neural network, a non-autoregressive gated recurrent unit, with other state-of-the-art black-box system identification models.

Our results show that autoregressive neural networks require significantly more time for training and inference than their non-autoregressive counterparts, without any benefits for accuracy. This limits the extent of possible hyperparameter optimization and model capacity, especially in more complex systems. Furthermore, we found that our implementation of non-autoregressive gated recurrent units outperforms all other neural network-based system identification models in the evaluated benchmark datasets and is among the best performing black-box models. In the future, we plan to publish our implementation of neural networks for system identification.

The present work focused on the simulation task, where no system output values are available for the model. In future work, the comparison may be done for the prediction task. While the one-step-ahead prediction task is trivial to implement non-autoregressively, the multi-step-ahead prediction is more challenging and may require a novel approach to avoid autoregression.

REFERENCES

Andersson, C., Ribeiro, A.H., Tiels, K., Wahlström, N., and Schön, T.B. (2019). Deep Convolutional Networks in System Identification. *arXiv:1909.01730 [cs, eess, stat]*.

Belz, J., Münker, T., Heinz, T.O., Kampmann, G., and Nelles, O. (2017). Automatic Modeling with Local Model Networks for Benchmark Processes. *IFAC-PapersOnLine*, 50(1), 470–475. doi:10.1016/j.ifacol.2017.08.089.

CHEN, S., BILLINGS, S.A., and GRANT, P.M. (1990). Non-linear system identification using neural networks. *International Journal of Control*, 51(6), 1191–1214. doi:10.1080/00207179008934126.

De Brabanter, K., Dreesen, P., Karsmakers, P., Pelckmans, K., De Brabanter, J., Suykens, J.A.K., and De Moor, B. (2009). Fixed-Size LS-SVM Applied to the Wiener-Hammerstein Benchmark. *IFAC Proceedings Volumes*, 42(10), 826–831. doi:10.3182/20090706-3-FR-2004.00137.

Gedon, D., Wahlström, N., Schön, T.B., and Ljung, L. (2020). Deep State Space Models for Nonlinear System Identification. *arXiv:2003.14162 [cs, eess, stat]*. ArXiv: 2003.14162.

Gonzalez, J. and Yu, W. (2018). Non-linear system modeling using LSTM neural networks. *IFAC-PapersOnLine*, 51(13), 485–489. doi:10.1016/j.ifacol.2018.07.326.

Hochreiter, S. (2011). The Vanishing Gradient Problem During Learning Recurrent Neural Nets and Problem Solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*. doi:10.1142/S0218488598000094. Publisher: World Scientific Publishing Company.

Howard, J. and Gugger, S. (2020). Fastai: A Layered API for Deep Learning. *Information*, 11(2), 108. doi:10.3390/info11020108.

Ioffe, S. and Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *arXiv:1502.03167 [cs]*.

Kumar, R., Srivastava, S., Gupta, J.R.P., and Mohindru, A. (2019). Comparative study of neural networks for dynamic nonlinear systems identification. *Soft Computing*, 23(1), 101–114. doi:10.1007/s00500-018-3235-5.

Lamb, A., Goyal, A., Zhang, Y., Zhang, S., Courville, A., and Bengio, Y. (2016). Professor Forcing: A New Algorithm for Training Recurrent Networks. *arXiv:1610.09038 [cs, stat]*. ArXiv: 1610.09038.

Lauwers, L. and Schoukens, J. (2010). Identification of Wiener–Hammerstein Systems Using the Best Linear Approximation. In M. Morari, M. Thoma, F. Giri, and E.W. Bai (eds.), *Block-oriented Nonlinear System Identification*, volume 404, 209–225. Springer London, London.

Li, L., Jamieson, K., Rostamizadeh, A., Gonina, E., Hardt, M., Recht, B., and Talwalkar, A. (2020). A System for Massively Parallel Hyperparameter Tuning. *arXiv:1810.05934 [cs, stat]*. ArXiv: 1810.05934.

Liu, L., Jiang, H., He, P., Chen, W., Liu, X., Gao, J., and Han, J. (2019). On the Variance of the Adaptive Learning Rate and Beyond. *arXiv:1908.03265 [cs, stat]*.

Ljung, L., Andersson, C., Tiels, K., and Schon, T.B. (2020). Deep Learning and System Identification. *Proc IFAC Congress*, 8.

Ljung, L., Zhang, Q., Lindskog, P., and Juditski, A. (2004). Estimation of grey box and black box models for nonlinear circuit data. *IFAC Proceedings Volumes*, 37(13), 399–404. doi:10.1016/S1474-6670(17)31256-9.

Lopez, M. and Yu, W. (2017). Nonlinear system modeling using convolutional neural networks. In *2017 14th International Conference on Electrical Engineering, Computing Science and Automatic Control (CCE)*, 1–5. IEEE, Mexico City. doi:10.1109/ICEEE.2017.8108894.

Loshchilov, I. and Hutter, F. (2017). SGDR: Stochastic Gradient Descent with Warm Restarts. *arXiv:1608.03983 [cs, math]*.

Maroli, J.M. and Redmill, K. (2019). Nonlinear System Identification Using Temporal Convolutional Networks: A Silverbox Study. *IFAC-PapersOnLine*, 52(29), 186–191. doi:10.1016/j.ifacol.2019.12.642.

Merity, S., Keskar, N.S., and Socher, R. (2017). Regularizing and Optimizing LSTM Language Models. *arXiv:1708.02182 [cs]*.

Oord, A.v.d., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., Kalchbrenner, N., Senior, A., and Kavukcuoglu, K. (2016). WaveNet: A Generative Model for Raw Audio. *arXiv:1609.03499 [cs]*.

Paduart, J., Lauwers, L., Pintelon, R., and Schoukens, J. (2009). Identification of a Wiener-Hammerstein System Using the Polynomial Nonlinear State Space Approach. *IFAC Proceedings Volumes*, 42(10), 1080–1085. doi:10.3182/20090706-3-FR-2004.00179.

Paduart, J., Lauwers, L., Swevers, J., Smolders, K., Schoukens, J., and Pintelon, R. (2010). Identification of nonlinear systems using Polynomial Nonlinear State Space models. *Automatica*, 46(4), 647–656. doi:10.1016/j.automatica.2010.01.001.

Paine, T.L., Khorrami, P., Chang, S., Zhang, Y., Ramachandran, P., Hasegawa-Johnson, M.A., and Huang, T.S. (2016). Fast Wavenet Generation Algorithm.

arXiv:1611.09482 [cs].

Ribeiro, A.H. and Aguirre, L.A. (2018). "Parallel Training Considered Harmful?": Comparing series-parallel and parallel feedforward network training. *Neurocomputing*, 316, 222–231. doi:10.1016/j.neucom.2018.07.071.

Romero Ugalde, H.M., Carmona, J.C., Alvarado, V.M., and Reyes-Reyes, J. (2013). Neural network design and model reduction approach for black box nonlinear system identification with reduced number of parameters. *Neurocomputing*, 101, 170–180. doi: 10.1016/j.neucom.2012.08.013.

Rosa, E.d.l., Yu, W., and Li, X. (2015). Nonlinear system identification using deep learning and randomized algorithms. In *2015 IEEE International Conference on Information and Automation*, 274–279. doi: 10.1109/ICInfA.2015.7279298.

Schoukens, J., Suykens, J., and Ljung, L. (2009). Wiener-Hammerstein Benchmark. In *15th IFAC Symposium on System Identification (SYSID 2009)*, 4.

Schoukens, M. and Noel, J.P. (2016). Wiener-Hammerstein benchmark with process noise. 5.

Schoukens, M. (2016). Identification of Wiener-Hammerstein systems with process noise using an Errors-in-Variables framework. *Workshop on Nonlinear System Identification Benchmarks*, 2.

Shi, G., Shi, X., O'Connell, M., Yu, R., Azizzadenesheli, K., Anandkumar, A., Yue, Y., and Chung, S.J. (2019). Neural Lander: Stable Drone Landing Control using Learned Dynamics. *arXiv:1811.08027 [cs]*.

Smith, L.N. (2017). Cyclical Learning Rates for Training Neural Networks. *arXiv:1506.01186 [cs]*.

Tallec, C. and Ollivier, Y. (2017). Unbiasing Truncated Backpropagation Through Time. *arXiv:1705.08209 [cs]*.

Weber, D., Gühmann, C., and Seel, T. (2020). Neural Networks Versus Conventional Filters for Inertial-Sensor-based Attitude Estimation. *arXiv:2005.06897 [cs, eess, stat]*. ArXiv: 2005.06897.

Wigren, T. and Schoukens, J. (2013). Three free data sets for development and benchmarking in nonlinear system identification. In *2013 European Control Conference (ECC)*, 2933–2938. doi:10.23919/ECC.2013.6669201.

Wills, A. and Ninness, B. (2009). Estimation of Generalised Hammerstein-Wiener Systems. *IFAC Proceedings Volumes*, 42(10), 1104–1109. doi:10.3182/20090706-3-FR-2004.00183.

Zadeh, L. (1956). On the Identification Problem. *IRE Transactions on Circuit Theory*, 3(4), 277–281. doi: 10.1109/TCT.1956.1086328. Conference Name: IRE Transactions on Circuit Theory.

Zhang, M.R., Lucas, J., Hinton, G., and Ba, J. (2019). Lookahead Optimizer: k steps forward, 1 step back. *arXiv:1907.08610 [cs, stat]*.